University of Colombo School of Computing
Master of Information Technology
MIT4301 — Independent Study

Research Project Management System (RPMS)

Background Analysis and System Design

Submitted by
D.M.P.G.C.K.B. Karunarathna
Index No: 24550337
Registration No: 2024/MIT/033
2026

Table of Contents

1.1 Introduction to the Topic 1
Why this topic 1
What is unfamiliar going in 1
Structure of this chapter 2
1.2 Background Analysis 2
1.2.1 The Problem Domain and Current Practice 2
Existing solutions 3
1.2.2 Stakeholder Analysis 3
1.2.3 System Requirements 4
Functional requirements 4
Non-functional requirements 5
1.2.4 Use Case Modelling 7
Use case narratives 8
1.3 System Design 14
1.3.1 System Architecture 14
Key architectural decisions 15
1.3.2 Detailed System Design 16
UML class diagram 16
State diagrams 17
1.3.3 Database Design 18
Entity relationship diagram 18
Relational schema 19
Normalisation and integrity 20
1.3.4 User Interface Design 20
Applying usability heuristics 23
1.4 Requirements Traceability 24
References 25

List of Figures

Figure 1.1: Use case diagram of the Research Project Management System 8
Figure 1.2: High-level architecture of RPMS 15
Figure 1.3: UML class diagram of the RPMS domain model 16
Figure 1.4: State machine diagram — Submission (proposal or research document) 17
Figure 1.5: State machine diagram — Project 18
Figure 1.6: Entity relationship diagram (crow's foot notation) 19
Figure 1.7: Wireframe — Login page 21
Figure 1.8: Wireframe — Student dashboard 21
Figure 1.9: Wireframe — Prepare and submit proposal (UC-01, UC-03) 22
Figure 1.10: Wireframe — Supervisor review of a submission (UC-02) 22
Figure 1.11: Wireframe — Submit progress update (UC-04) 23
Figure 1.12: Wireframe — Coordinator project management (UC-05, UC-06) 23

List of Tables

Table 1.1: Problems in the current practice and the RPMS response 2
Table 1.2: Stakeholders of RPMS 3
Table 1.3: Functional requirements 4
Table 1.4: Non-functional requirements 6
Table 1.5: Actors of RPMS 7
Table 1.6: Application of Nielsen's usability heuristics 24
Table 1.7: Traceability from use cases to requirements, data and interface 24

1.1 Introduction to the Topic

A student research project is a long-running process that progresses through several stages, from the initial selection of a research topic to the final submission and completion of the project. During this process, students are required to prepare and revise their research proposals, define and track project milestones, submit different versions of chapters and other project documents, receive feedback from their supervisors, and provide regular progress updates.

As the project progresses, both the student and supervisor need a clear and up-to-date view of the project's current status. In addition, the academic coordinator requires an overall view of the progress of research projects under their coordination in order to monitor their development effectively.

The **Research Project Management System (RPMS)** is a role-based web application developed to support and manage this workflow through a centralized platform. The system provides separate functionality for students, supervisors, and the academic coordinator to manage research projects, proposals, milestones, document submissions and their versions, progress updates, and feedback. The system maintains project-related information in a persistent database**,** allowing relevant information to be accessed and updated throughout the project lifecycle.

Why this topic

My motivation for selecting this topic comes from my own experience with my undergraduate research. Managing different versions of proposals, identifying which version had been reviewed by the supervisor, keeping track of feedback associated with each draft, and remembering the next tasks to be completed often involved using a combination of email conversations, chat messages, and file names such as *proposal_final_v3_revised.docx*.

Although each of these tools is useful for its intended purpose, they do not provide a single, connected view of the research project. As a result, the current status of a project often has to be reconstructed by referring to different messages, files, and previous discussions.

This experience motivated me to develop RPMS as a focused solution to this problem. The system brings project documents, versions, feedback, milestones, and progress information together in one place, making the research process easier to organize and monitor. Since I have experienced these challenges from the student's perspective, I also have a practical understanding of the problem that the system is intended to address.

What is unfamiliar going in

Although I write software professionally, PHP with MySQL on XAMPP is not the stack I use day to day, so setting it up and structuring a PHP application cleanly without a framework is new to me.

I also expect to learn most from the design problems that are specific to this domain: storing every submitted version without overwriting earlier ones, linking feedback to the exact version it refers to, and enforcing on the server that each user can only see the projects they are entitled to see.

Structure of this chapter

Section 1.2 analyses the problem domain and identifies the key stakeholders, functional requirements, and non-functional requirements of the system. It also models the required system behavior using a use case diagram and detailed use case narratives.

Section 1.3 presents the proposed solution design, including the system architecture, class and state diagrams, database design through the Entity-Relationship Diagram (ERD) and relational schema, and wireframes for the main user interfaces.

Section 1.4 traces the identified requirements through the proposed design, demonstrating how the system design addresses the requirements established during the analysis.

1.2 Background Analysis

1.2.1 The Problem Domain and Current Practice

In many departments, research supervision is managed informally. Proposals and chapter drafts are exchanged as e-mail attachments, feedback is given as tracked changes, e-mail replies or conversations, milestones are agreed verbally or recorded in a spreadsheet kept by the coordinator, and progress reports are sent as free-form messages. This works for a single student and supervisor, but it scales poorly and leaves no reliable record. Table 1.1 summarizes the problems that follow from this practice and how RPMS is intended to address each of them.

Table 1.1: *Problems in the current practice and the RPMS response*

Problem in current practice	Consequence	How RPMS addresses it
Documents exchanged as e-mail attachments with ad-hoc file names	Uncertainty over which version is the latest, and which version a comment refers to	Every submission is stored as a numbered version that is never overwritten; feedback is linked to a specific version
Feedback scattered across e-mails, chats and meetings	Comments are lost or forgotten; students cannot show how they responded	All feedback is stored with the submission or progress update it belongs to, in one feedback history
No shared view of project status	Supervisors and the coordinator must ask students where they are	Each project and submission has an explicit status; role-based dashboards show it at a glance
Milestones kept in personal notes or a coordinator's spreadsheet	Deadlines slip without anyone noticing	Milestones with due dates are defined per project and flagged when overdue
Progress reported irregularly and informally	Problems surface late, often close to deadlines	Structured progress updates (work done, challenges, next steps) that the supervisor reviews
No audit trail	Disputes about what was submitted or agreed are hard to resolve	Timestamps on versions, feedback and updates provide a traceable history

Existing solutions

General-purpose tools each cover part of this workflow. Learning management systems such as Moodle provide assignment submission and feedback, but they are organized around courses with fixed deadlines and a whole cohort, rather than a long-running one-to-one supervision relationship with repeated revise-and-resubmit cycles.

Task boards such as Trello make milestones visible but have no concept of formal submission, approval or versioned documents. Shared cloud folders keep files in one place but not the review status or the feedback trail.

RPMS addresses this gap by bringing together the key elements required for student research supervision, including research projects, proposals, versioned document submissions, feedback, milestones, and progress updates, within a single role-based system. The scope is intentionally limited to the core requirements of research project management, making the system sufficiently focused and manageable for development within the scope of this module.

1.2.2 Stakeholder Analysis

A stakeholder is any person or group who interacts with the system or is affected by it (Sommerville, 2016). Table 1.2 identifies the stakeholders of RPMS, distinguishing *primary* stakeholders, who use the system directly and appear as actors in the use case model, from *secondary* stakeholders, who are affected by the system or support it without using its main functions.

Table 1.2: *Stakeholders of RPMS*

Stakeholder	Type	Interest in / role with the system	Influence / Interest
Student (researcher)	Primary (actor)	Creates the project, prepares and submits the proposal, uploads documents and revised versions, submits progress updates, and reads feedback. Wants a clear view of what is expected next and proof of what was submitted.	Medium / High
Supervisor	Primary (actor)	Reviews proposals, documents and progress updates for assigned students, records feedback, requests revisions and approves submissions. Wants to monitor several students without searching through e-mail.	High / High
Academic Coordinator / Administrator	Primary (actor)	Manages user accounts, creates projects, assigns supervisors, defines milestones and monitors the status of all projects. Wants an overview of all projects and early warning of stalled work.	High / High
Developer	Secondary	Designs, builds, tests and maintains the system.	High / High

The three primary stakeholders have both high interest and direct influence over whether the system is useful, so their needs drive the functional requirements.

1.2.3 System Requirements

Functional requirements

Functional requirements describe what the system must do. They are grouped by module and prioritised using the MoSCoW scheme (**M**ust, **S**hould, **C**ould have). *Must* requirements together form the minimum viable version agreed at the proposal stage, while *Should* and *Could* requirements define additional functionality that will be implemented as part of the system.

Table 1.3: *Functional requirements*

ID	Requirement	Actor	Priority
Authentication and user management			
FR-01	The system shall authenticate users with an e-mail address and password.	All	M
FR-02	The system shall restrict each function and each record to the roles and users entitled to it (for example, a student can only view their own project; a supervisor only projects assigned to them).	All	M
FR-03	The system shall allow users to log out and shall end idle sessions automatically.	All	M
FR-04	The coordinator shall be able to create, edit and deactivate user accounts and assign each account one role.	Coordinator	M
FR-05	A user shall be able to change their own password.	All	S
Project management			
FR-06	A student shall be able to create a research project with a title, description and research area.	Student	M
FR-07	The coordinator shall be able to create a project for a student and edit project details.	Coordinator	S
FR-08	The coordinator shall be able to assign and reassign a supervisor to a project; assignment moves a new project to the PROPOSAL_STAGE status.	Coordinator	M
FR-09	The coordinator shall be able to mark a project as COMPLETED once all its milestones are completed.	Coordinator	S
Proposal and document submission			
FR-10	A student shall be able to prepare a proposal (title, summary and document file) and save it as a draft that is not visible to the supervisor.	Student	M
FR-11	A student shall be able to submit the proposal for review once a supervisor is assigned.	Student	M
FR-12	A student shall be able to upload research documents (e.g. chapter drafts), optionally linked to a milestone.	Student	M
FR-13	The system shall store every submission and resubmission as a new, numbered version and shall never overwrite an earlier version.	System	M

ID	Requirement	Actor	Priority
FR-14	The system shall accept only PDF and DOCX files up to 10 MB.	System	M
FR-15	A student shall be able to submit a revised version only when the supervisor has requested a revision, with a note describing the changes.	Student	M
FR-16	Authorised users shall be able to download any version of a submission.	Student, Supervisor	M
Review and feedback			
FR-17	A supervisor shall see a review queue of submitted items and unreviewed progress updates for their assigned projects.	Supervisor	M
FR-18	A supervisor shall record feedback on a specific version with one decision: comment only, request revision, or approve.	Supervisor	M
FR-19	Approving a proposal shall change the project status to ACTIVE.	System	M
FR-20	A student shall be able to view all feedback, grouped by the version or progress update it refers to.	Student	M
Milestones and progress			
FR-21	The coordinator shall be able to define milestones (title, description, due date) for a project.	Coordinator	M
FR-22	The supervisor shall be able to update the status of a milestone (PENDING, IN_PROGRESS, COMPLETED).	Supervisor	M
FR-23	The system shall flag milestones that are past their due date and not completed as overdue.	System	S
FR-24	A student shall be able to submit a progress update (work completed, challenges, next steps), optionally linked to a milestone.	Student	M
FR-25	A supervisor shall be able to review a progress update, add feedback and mark it as reviewed.	Supervisor	M
Dashboards and status			
FR-26	Each role shall have a dashboard: the student sees project status, milestones, pending actions and recent feedback; the supervisor sees the review queue and assigned projects; the coordinator sees all projects by status, including unassigned ones.	All	M
FR-27	The coordinator shall be able to search and filter projects by status and supervisor.	Coordinator	C

Non-functional requirements

Non-functional requirements constrain how the system delivers its functions. They are organised using the quality characteristics of ISO/IEC 25010 (ISO/IEC, 2011), and each has a concrete way of being verified so that it can be tested rather than merely claimed.

Security receives the most attention because RPMS stores unpublished research and assessment feedback, and because the most common web application weaknesses, such as broken access control, injection and cross-site scripting, are directly relevant to a PHP and MySQL application (OWASP, 2021).

Table 1.4: *Non-functional requirements*

ID	Category	Requirement	Verification
NFR-01	Security — authentication	Passwords are stored only as salted hashes using PHP's password_hash() (bcrypt) and checked with password_verify().	Inspect users table; no plain-text passwords
NFR-02	Security — access control	Access rules are enforced on the server for every request, including object-level checks (ownership / assignment), not only by hiding menu items.	Attempt to open another user's project by editing the URL; expect HTTP 403
NFR-03	Security — injection	All database access uses PDO prepared statements with bound parameters.	Code review; SQL injection test strings in forms
NFR-04	Security — XSS and CSRF	All user-supplied output is escaped with htmlspecialchars(); every state-changing form carries a CSRF token.	Enter script tags in text fields; submit a form without a token
NFR-05	Security — sessions	The session ID is regenerated at login and sessions expire after 30 minutes of inactivity.	Functional test of login and idle timeout
NFR-06	Security — file upload	Uploaded files are checked by extension and MIME type, limited to 10 MB, renamed with a random name and stored outside the web root; they are served only through an access-checked download controller.	Upload a renamed .php file; try to access storage directly
NFR-07	Usability	The interface follows a consistent layout and navigation per role; validation errors are shown next to the field in plain language; the main task of each role is reachable within three clicks of the dashboard.	Heuristic review (Nielsen, 1994); task walkthroughs
NFR-08	Usability — responsiveness	Pages remain usable on screens from 360 px wide (Bootstrap grid).	Test in browser device emulation
NFR-09	Performance efficiency	Each page loads within 2 seconds on the local environment with test data of 50 users and 100 projects.	Browser developer tools timing
NFR-10	Reliability — integrity	Foreign keys and CHECK constraints protect referential integrity; operations that change several rows (e.g. storing a new version and updating status) run in a single transaction.	Force an error mid-operation; confirm no partial records
NFR-11	Traceability	Submitted versions and feedback cannot be edited or deleted by users; all are time-stamped.	Attempt edit after submission

ID	Category	Requirement	Verification
NFR-12	Maintainability	Code is organised into a front controller, controllers, models and views; follows PSR-12 style; developed with small, meaningful Git commits.	Repository history; code review
NFR-13	Portability	The system runs on a standard XAMPP installation and on current versions of Chrome, Firefox and Edge.	Fresh install from the repository
NFR-14	Recoverability	The database can be backed up and restored with a documented mysqldump / phpMyAdmin export procedure.	Backup and restore test

1.2.4 Use Case Modelling

Use case modelling describes the system from the point of view of its users: the *actors* that interact with the system and the goals (*use cases*) they achieve through it (Fowler, 2004). Table 1.5 describes the actors.

Table 1.5: *Actors of RPMS*

Actor	Description
User (abstract)	Any authenticated person. Generalises the three concrete actors and holds the use cases common to all of them: Log in and View Dashboard & Project Status.
Student	A postgraduate or undergraduate researcher who owns one research project and produces the proposal, documents and progress updates.
Supervisor	An academic staff member assigned to one or more projects, who reviews submissions and progress updates and gives feedback.
Academic Coordinator	The administrator of the research programme, who manages accounts, projects, supervisor assignment and milestones.

Figure 1.1 shows the use case diagram. The *User* actor is used as a generalisation so that the common use cases are drawn once rather than connected to every actor.

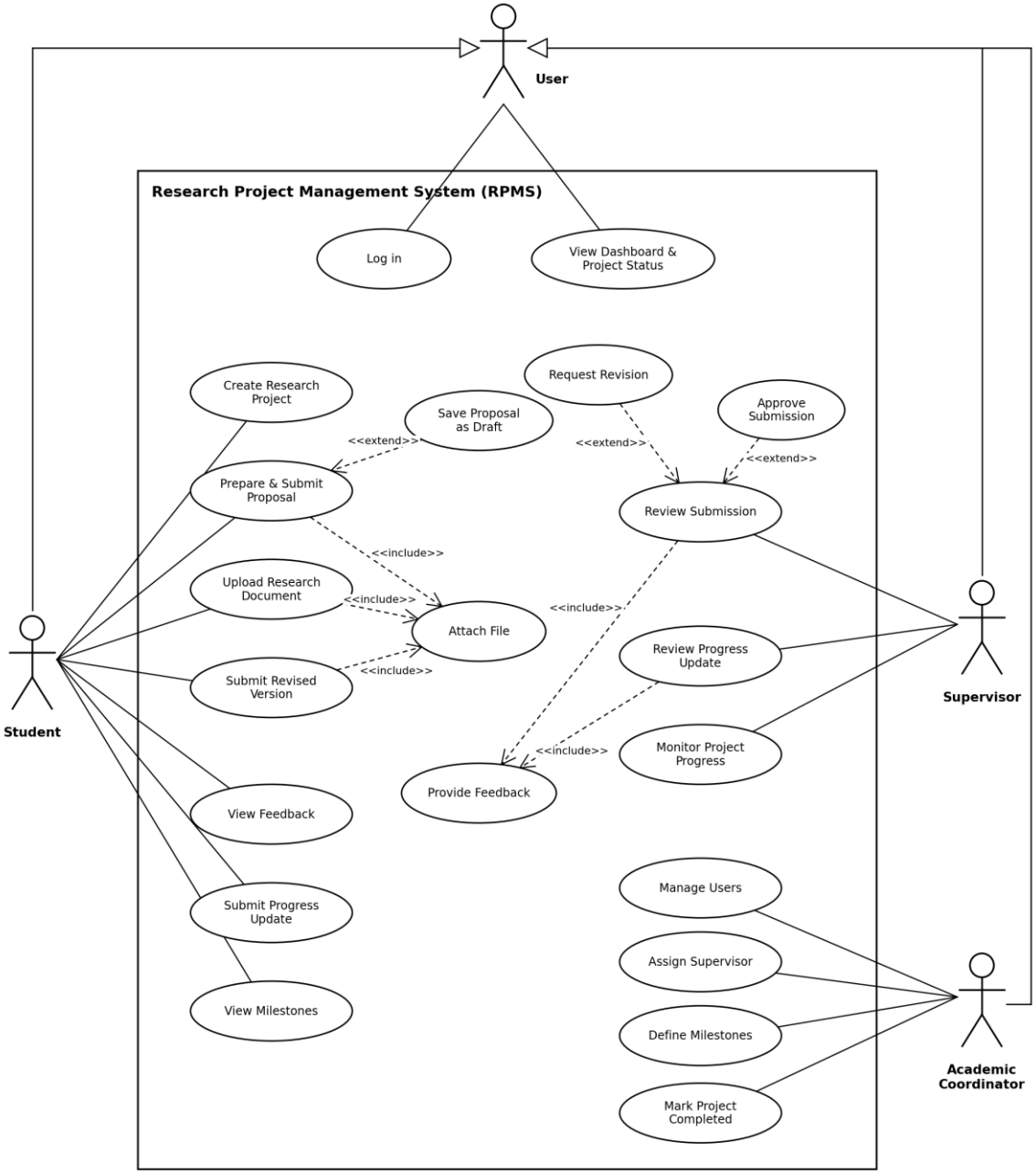


Figure 1.1: *Use case diagram of the Research Project Management System*

Use case narratives

Narratives are written for the six use cases that carry the core workflow and the most important business rules. The format follows the structured, fully-dressed style described by Cockburn (2001).

Use Case ID	UC-01
Use Case Name	Prepare & Submit Proposal
Primary Actor	Student
Secondary Actor	Supervisor (receives the submission for review)
Goal	Prepare a research proposal, optionally save it as a draft, and submit it to the assigned supervisor for review.
Preconditions	1. The student is logged in. 2. The student's project exists and has status PROPOSAL_STAGE (a supervisor is assigned). 3. The project does not already have an approved proposal.
Postconditions	**Success:** a proposal submission exists with status SUBMITTED; version 1 is stored with its submission time and appears in the supervisor's review queue. **Draft:** the proposal is stored with status DRAFT and is not visible to the supervisor.
Includes / Extends	Includes: Attach File Extended by: Save Proposal as Draft (extension point: before submission)
Main Success Flow	1. Student selects *Proposal* from the navigation bar. 2. System displays the proposal form, pre-filled if a draft exists. 3. Student enters the proposal title and summary. 4. System executes <<include>> Attach File; student selects the proposal document. 5. System validates the required fields and the file type and size. 6. Student selects *Submit*. 7. System, in one transaction, stores the file under a generated name, records version 1 with the submission time and sets the status to SUBMITTED. 8. System displays a confirmation, and the dashboard shows the proposal as SUBMITTED.
Extensions / Exceptions	2a. **No supervisor assigned:** System disables submission and displays "A supervisor must be assigned before the proposal can be submitted"; saving a draft remains available. 5a. **Validation fails:** System highlights missing fields or rejects the file with "Only PDF or DOCX files up to 10 MB are accepted"; flow returns to step 3. 6a. **Save as draft (<<extend>> Save Proposal as Draft):** 1. Student selects *Save as draft*. 2. System stores the data (file optional) with status DRAFT and no submission time. 3. Use case ends; the student can resume later from step 2. 7a. **Storage or database error:** System rolls back the transaction, deletes any stored file and shows an error; no partial record remains.

Use Case ID	UC-02
Use Case Name	Review Submission
Primary Actor	Supervisor
Secondary Actor	Student (receives feedback)
Goal	Review a submitted proposal or research document version and record feedback with a decision.
Preconditions	1. The supervisor is logged in. 2. The submission belongs to a project assigned to this supervisor. 3. The submission status is SUBMITTED.
Postconditions	Feedback is stored and linked to the reviewed version; the submission status is unchanged (comment only), REVISION_REQUESTED or APPROVED; if an approved submission is a proposal, the project becomes ACTIVE.
Includes / Extends	Includes: Provide Feedback Extended by: Request Revision, Approve Submission (extension point: decision)
Main Success Flow	1. Supervisor opens the *Review Queue*. 2. System lists SUBMITTED items and unreviewed progress updates for the supervisor's projects only. 3. Supervisor selects a submission. 4. System displays the submission details, the latest version (summary, change notes, download link) and the feedback history. 5. Supervisor downloads and reads the document. 6. System executes <<include>> Provide Feedback; supervisor enters a comment and selects *Comment only*. 7. System stores the feedback (decision COMMENT) linked to the version; the status stays SUBMITTED. 8. System confirms, and the feedback becomes visible to the student.
Extensions / Exceptions	3a. **Unauthorised access:** if the supervisor opens a submission of a project not assigned to them (e.g. by editing the URL), System returns HTTP 403 and shows no data. 6a. **Request revision (<<extend>> Request Revision):** supervisor selects *Request revision*; System stores the feedback with decision REVISION_REQUESTED, sets the submission status to REVISION_REQUESTED and shows an "Action needed" notice on the student's dashboard. 6b. **Approve (<<extend>> Approve Submission):** supervisor selects *Approve*; System stores the feedback with decision APPROVED and sets the status to APPROVED (read-only). If the submission is a proposal, System sets the project status to ACTIVE in the same transaction. 6c. **Empty comment:** System rejects the feedback; a comment is required for every decision.

Use Case ID	UC-03
Use Case Name	Submit Revised Version
Primary Actor	Student
Goal	Respond to a supervisor's revision request by submitting a new version of the proposal or document.
Preconditions	1. The student is logged in. 2. The submission belongs to the student's project. 3. The submission status is REVISION_REQUESTED.
Postconditions	A new version (previous number + 1) is stored with its submission time; the submission status is SUBMITTED; all earlier versions and their feedback remain unchanged.
Includes / Extends	Includes: Attach File
Main Success Flow	1. Student selects the "Action needed" item on the dashboard. 2. System displays the latest feedback and the version history. 3. Student selects *Submit revised version*. 4. Student updates the summary and enters notes describing what changed. 5. System executes <<include>> Attach File; student selects the revised document. 6. System validates the fields and the file. 7. System, in one transaction, stores version n + 1, updates the current version number and sets the status to SUBMITTED. 8. System confirms; the item reappears in the supervisor's review queue.
Extensions / Exceptions	3a. **Status is not REVISION_REQUESTED:** the option is not displayed, and a direct request to the URL is rejected by the server. 6a. **Validation fails:** as UC-01 5a. 7a. **Storage or database error:** as UC-01 7a.

Use Case ID	UC-04
Use Case Name	Submit Progress Update
Primary Actor	Student
Secondary Actor	Supervisor (reviews the update in *Review Progress Update*)
Goal	Report progress for the current period so that the supervisor can monitor the project and respond to problems early.
Preconditions	1. The student is logged in. 2. The student's project has status ACTIVE.
Postconditions	A progress update is stored with its submission time and marked as not reviewed; it appears in the supervisor's review queue.
Includes / Extends	—
Main Success Flow	1. Student selects *Progress Updates → New update*. 2. System displays the form and a list of the project's milestones that are not yet completed. 3. Student optionally selects the related milestone. 4. Student enters the work completed, challenges and next steps. 5. Student selects *Submit update*. 6. System validates that work completed and next steps are provided. 7. System stores the progress update. 8. System confirms and shows the update in the history as NOT REVIEWED.
Extensions / Exceptions	2a. **Project not ACTIVE:** System explains that progress updates become available after the proposal is approved. 6a. **Validation fails:** System highlights the missing fields; flow returns to step 4.

Use Case ID	UC-05
Use Case Name	Assign Supervisor
Primary Actor	Academic Coordinator
Secondary Actor	Supervisor, Student (informed through their dashboards)
Goal	Assign, or reassign, a supervisor to a research project.
Preconditions	1. The coordinator is logged in. 2. The project exists with status NEW or PROPOSAL_STAGE. 3. At least one active supervisor account exists.
Postconditions	The project records the selected supervisor; a NEW project moves to PROPOSAL_STAGE; the project appears on the supervisor's dashboard.
Includes / Extends	—
Main Success Flow	1. Coordinator opens *Projects*. 2. System lists projects with filters; projects without a supervisor are highlighted. 3. Coordinator selects a project. 4. System lists active supervisors with the number of projects each currently supervises. 5. Coordinator selects a supervisor and confirms. 6. System stores the assignment and sets the status to PROPOSAL_STAGE if it was NEW. 7. System confirms the assignment.
Extensions / Exceptions	3a. **Reassignment:** the project already has a supervisor; System asks for confirmation. Existing submissions and feedback are kept, and the new supervisor can see the full history. 3b. **Project COMPLETED:** assignment is disabled. 4a. **No active supervisors:** System displays a message with a link to Manage Users.

Use Case ID	UC-06
Use Case Name	Define Milestones
Primary Actor	Academic Coordinator
Goal	Define the milestones, with due dates, against which a project's progress is tracked.
Preconditions	1. The coordinator is logged in. 2. The project has status PROPOSAL_STAGE or ACTIVE.
Postconditions	The milestone is stored with status PENDING and is visible to the student and the supervisor.
Includes / Extends	—
Main Success Flow	1. Coordinator opens the project. 2. Coordinator selects *Define milestone*. 3. Coordinator enters the title, description and due date. 4. System validates that the title is present and that the due date is not in the past and not after the project end date (if set). 5. System stores the milestone with status PENDING. 6. System refreshes the milestone list, ordered by due date.
Extensions / Exceptions	4a. **Validation fails:** System shows the error next to the field; flow returns to step 3. 5a. **Editing an existing milestone:** the coordinator can edit any milestone; a milestone already referenced by submissions or progress updates cannot be deleted.

1.3 System Design

1.3.1 System Architecture

RPMS uses a classic three-tier web architecture (Figure 1.2). The **presentation tier** is the user's web browser, which renders HTML pages styled with Bootstrap and uses JavaScript only for client-side validation and small interface behaviours. The **application tier** is a PHP application running on the Apache web server provided by XAMPP. The **data tier** consists of the MySQL database and a file store for uploaded documents.

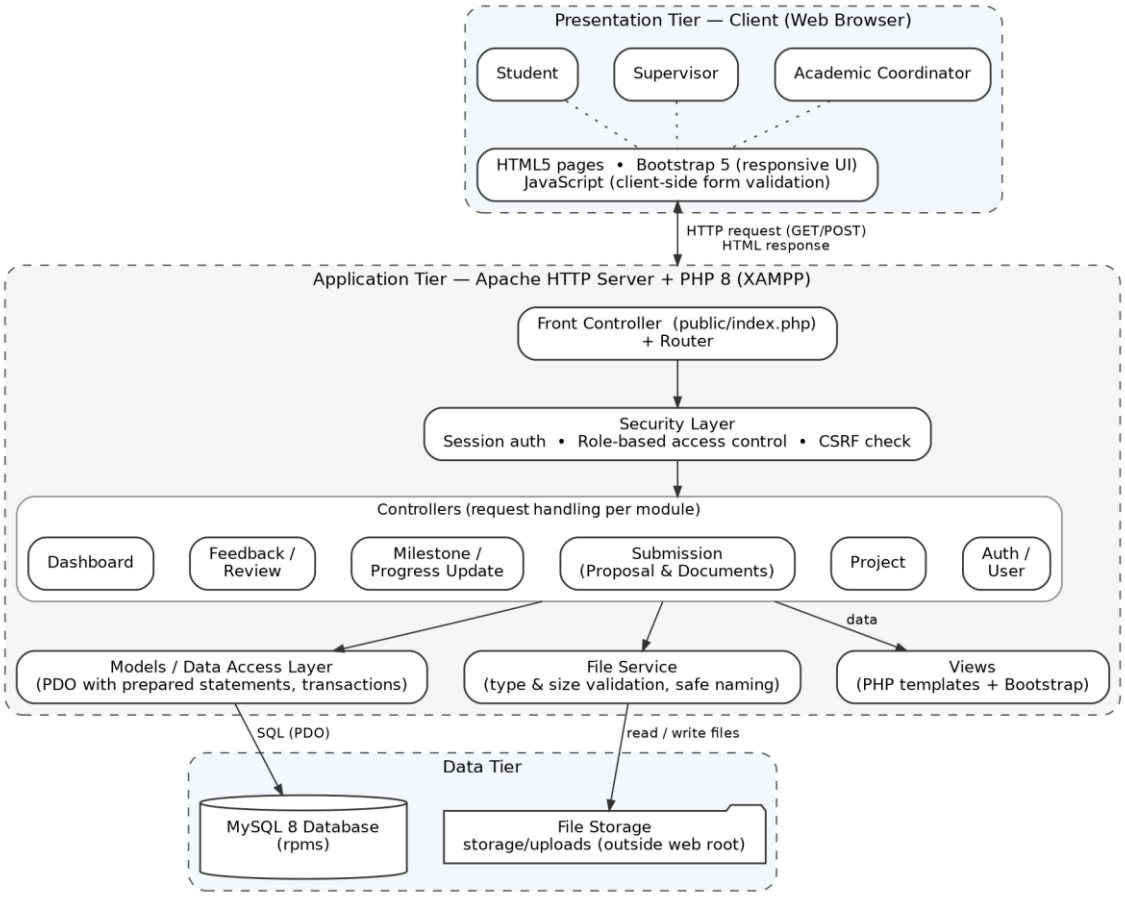


Figure 1.2: *High-level architecture of RPMS*

Every request enters the application through a single **front controller** (*public/index.php*), which passes it to a small router. Before any controller runs, a **security layer** checks that the user is logged in, that their role may use the requested function and, for form submissions, that the CSRF token is valid. The **controllers** — one per module — handle the request, call the **models** for data access and the **file service** for uploads and downloads, and pass the results to a **view** that produces the HTML response. The models use PDO with prepared statements and wrap multi-step changes in transactions.

Key architectural decisions

- **MVC structure.**

  The system follows the Model–View–Controller (MVC) architecture to separate data management, user interface, and application logic. Models handle data and database operations, Views are responsible for presenting information to users, and Controllers handle requests and coordinate the application logic. This separation improves maintainability, code organization, and ease of future development.

- **Front controller and a single security layer.**

  Putting authentication, role checks and CSRF checks in one place means they cannot be forgotten on an individual page, which directly supports NFR-02 and NFR-04.

- **Uploaded files outside the web root.**

  Documents are stored in *storage/uploads*, which Apache does not serve directly; every download goes through a controller that checks access first (NFR-06).

1.3.2 Detailed System Design

UML class diagram

Figure 1.3 shows the domain classes, their attributes and main operations, and the associations between them.

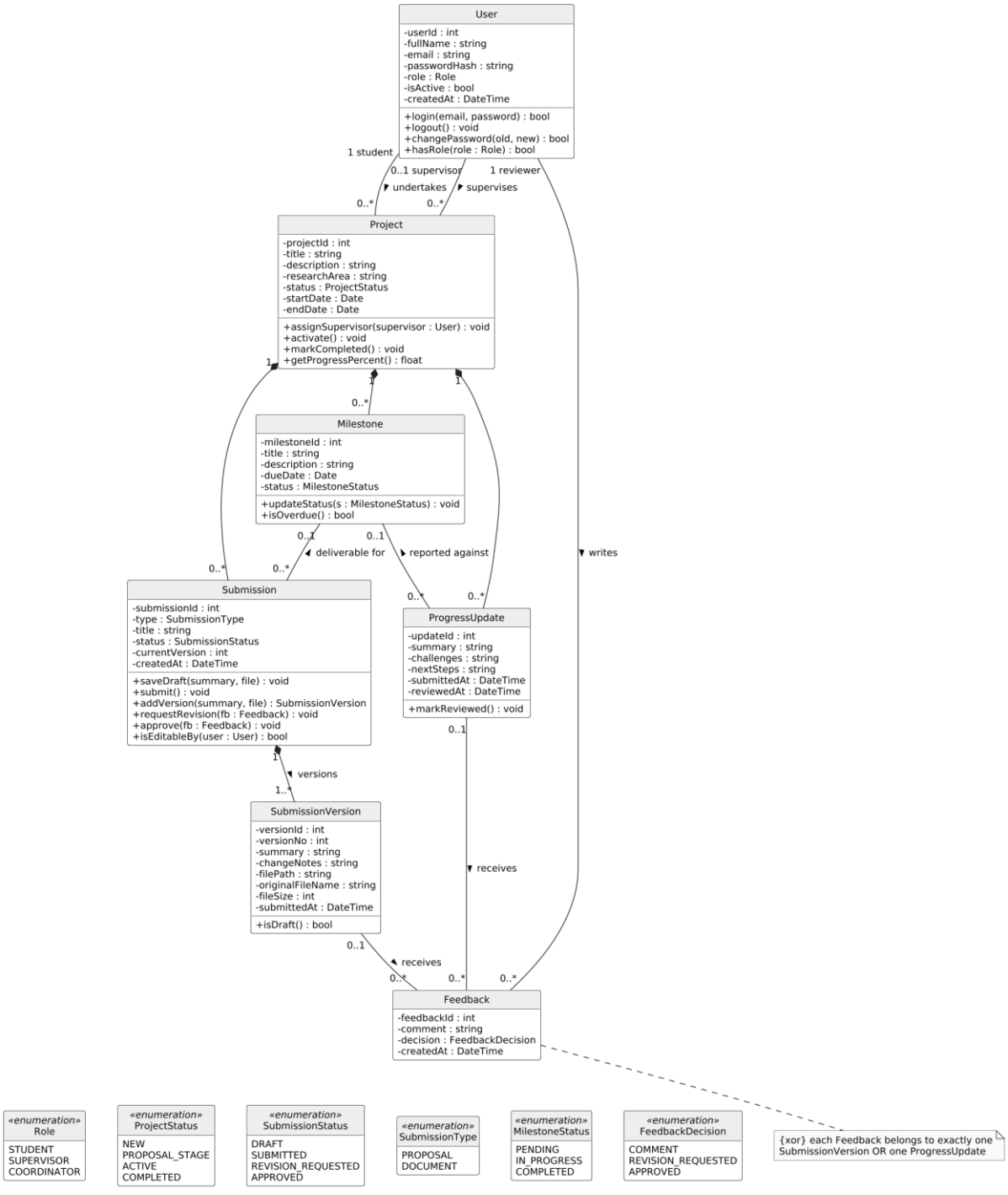


Figure 1.3: *UML class diagram of the RPMS domain model*

The main design decisions:

- **One User class with a Role.** Students, supervisors and coordinators share the same attributes and differ only in what they are allowed to do. Role-specific behaviour lives in the access-control layer, not in the class hierarchy.

- **Two associations between User and Project.** A project has exactly one student and zero or one supervisor (none while it is NEW).
- **One Submission class for proposals and documents.** Proposals and research documents follow the same submit–review–revise–approve cycle, so they share one *Submission* class distinguished by *type*.
- **Submission and SubmissionVersion (composition).** A submission is the logical item ("Research Proposal"); each upload is an immutable *SubmissionVersion*. This is how FR-13 (never overwrite) and FR-18 (feedback on a specific version) are met. The submission keeps *currentVersion* for quick access to the latest version.
- **Feedback with an exclusive target.** A feedback item belongs either to one submission version or to one progress update, never both and never neither.

State diagrams

Two classes have lifecycles that drive the business rules, so state machine diagrams are drawn for them. Figure 1.4 shows the lifecycle of a **Submission**. Only proposals can start in the DRAFT state; research documents are submitted directly. A supervisor's comment without a decision leaves the submission in SUBMITTED. A revision request moves it to REVISION_REQUESTED, and the only way out of that state is for the student to submit a new version, which returns it to SUBMITTED with an incremented version number. APPROVED is final: the submission becomes read-only.

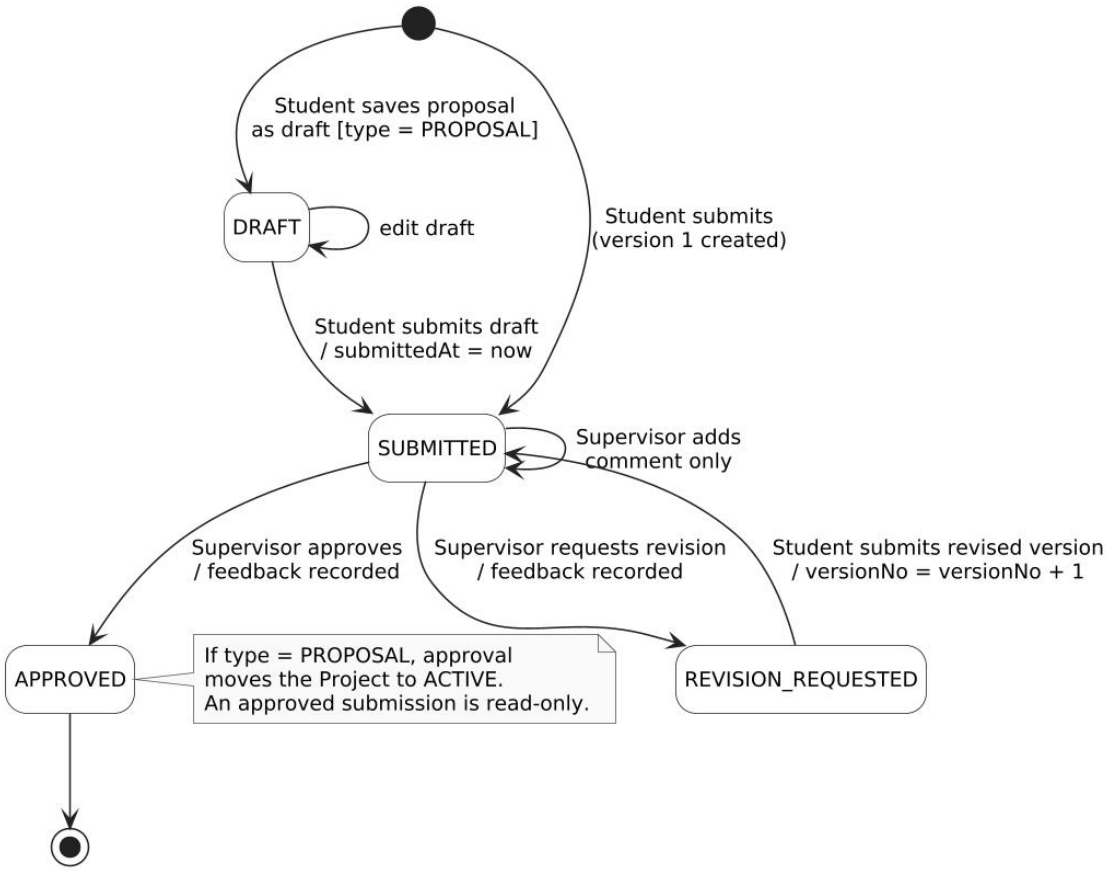


Figure 1.4: *State machine diagram — Submission (proposal or research document)*

Figure 1.5 shows the lifecycle of a **Project**. A project starts as NEW and moves to PROPOSAL_STAGE when a supervisor is assigned. It becomes ACTIVE automatically when its proposal is approved (the transition triggered by the Submission state machine), and it is

marked COMPLETED by the coordinator only when all milestones are completed. These states also control which functions are available; for example, progress updates are only accepted in the ACTIVE state (UC-04, extension 2a).

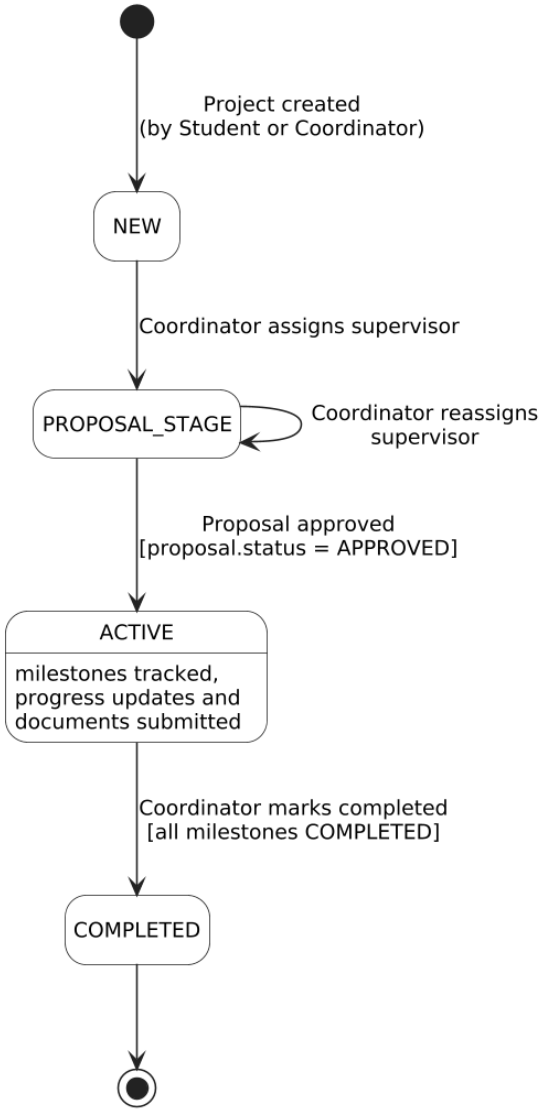


Figure 1.5: *State machine diagram — Project*

1.3.3 Database Design

Entity relationship diagram

Figure 1.6 shows the entity relationship diagram in crow's foot notation. Seven entities are required. The **users** entity is related to **projects** twice (as the student and as the supervisor) and to **feedback** as the reviewer. Each project has many milestones, submissions and progress updates. Each submission has one or more versions, and feedback is attached to either a version or a progress update. For readability, the audit relationships from *created_by* in projects and milestones to users are not drawn.

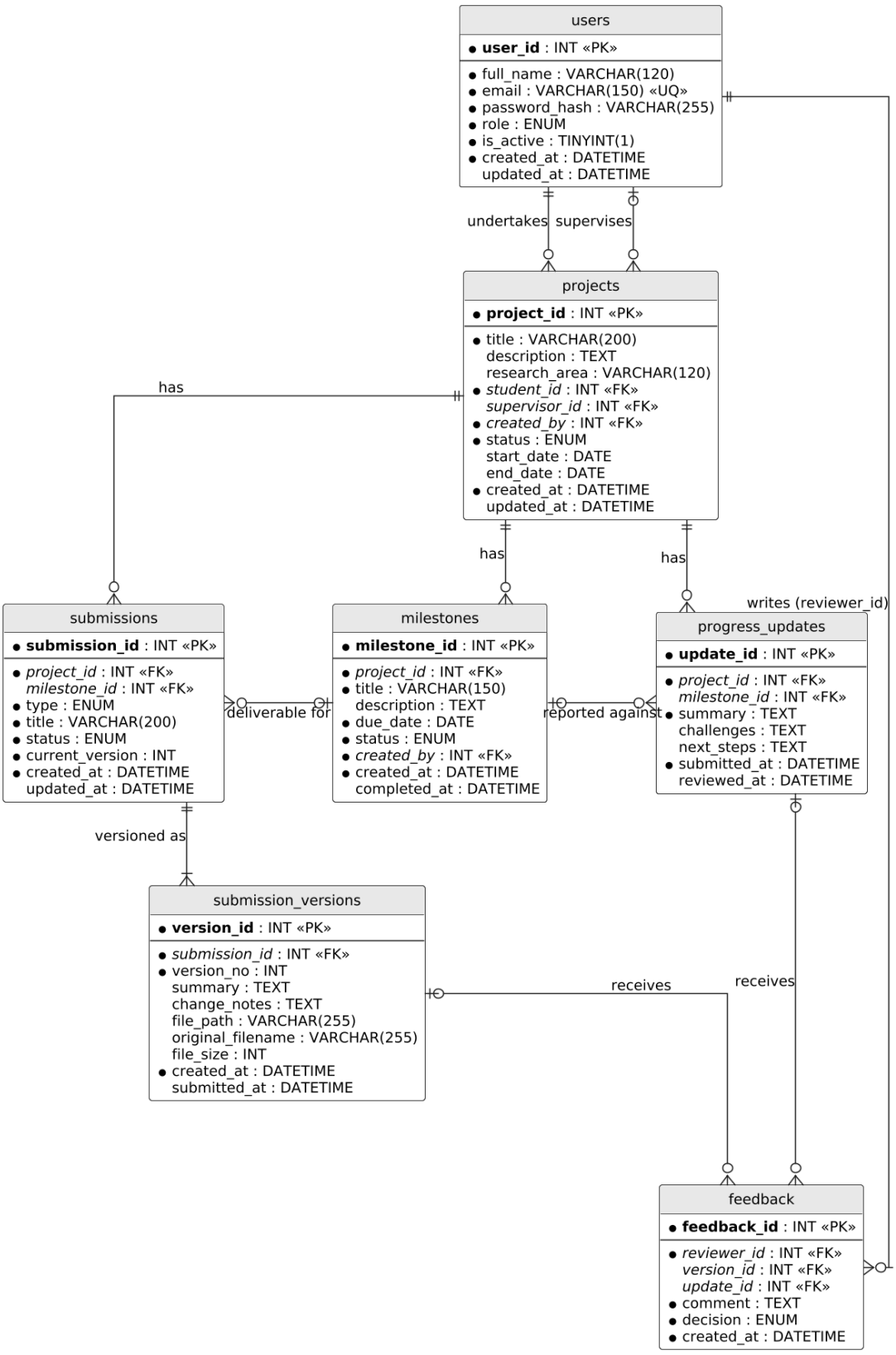


Figure 1.6: *Entity relationship diagram (crow's foot notation)*

Relational schema

The ERD maps to the following relational schema. Primary keys are underlined and foreign keys are shown in *italics*, with the table each one references listed underneath.

USERS (user_id, full_name, email, password_hash, role, is_active, created_at, updated_at)

email is UNIQUE; role ∈ {STUDENT, SUPERVISOR, COORDINATOR}

PROJECTS (project_id, title, description, research_area, *student_id*, *supervisor_id*, *created_by*, status, start_date, end_date, created_at, updated_at)

student_id, supervisor_id (nullable), created_by → users(user_id); status ∈ {NEW, PROPOSAL_STAGE, ACTIVE, COMPLETED}

MILESTONES (milestone_id, *project_id*, title, description, due_date, status, *created_by*, created_at, completed_at)

project_id → projects(project_id); created_by → users(user_id); status ∈ {PENDING, IN_PROGRESS, COMPLETED}

SUBMISSIONS (submission_id, *project_id*, *milestone_id*, type, title, status, current_version, created_at, updated_at)

project_id → projects(project_id); milestone_id (nullable) → milestones(milestone_id); type ∈ {PROPOSAL, DOCUMENT}; status ∈ {DRAFT, SUBMITTED, REVISION_REQUESTED, APPROVED}

SUBMISSION_VERSIONS (version_id, *submission_id*, version_no, summary, change_notes, file_path, original_filename, file_size, created_at, submitted_at)

submission_id → submissions(submission_id); (submission_id, version_no) is UNIQUE; submitted_at is NULL while the version is a draft

PROGRESS_UPDATES (update_id, *project_id*, *milestone_id*, summary, challenges, next_steps, submitted_at, reviewed_at)

project_id → projects(project_id); milestone_id (nullable) → milestones(milestone_id)

FEEDBACK (feedback_id, *reviewer_id*, *version_id*, *update_id*, comment, decision, created_at)

reviewer_id → users(user_id); version_id → submission_versions(version_id); update_id → progress_updates(update_id); CHECK: exactly one of version_id and update_id is not NULL; decision ∈ {COMMENT, REVISION_REQUESTED, APPROVED}

Normalisation and integrity

Every table has a single-attribute surrogate primary key, every non-key attribute depends only on that key, and no non-key attribute depends on another non-key attribute, so the schema is in third normal form (Elmasri and Navathe, 2016). For example, a supervisor's name is not repeated in projects; it is obtained by joining to users through supervisor_id. One deliberate exception is *submissions.current_version*, which could be derived as the highest version_no; it is stored because it is read on almost every page, and it is only ever updated in the same transaction that inserts a new version, so it cannot become inconsistent.

1.3.4 User Interface Design

Low-fidelity wireframes were produced for the six screens that support the use cases described in Section 1.2.4.

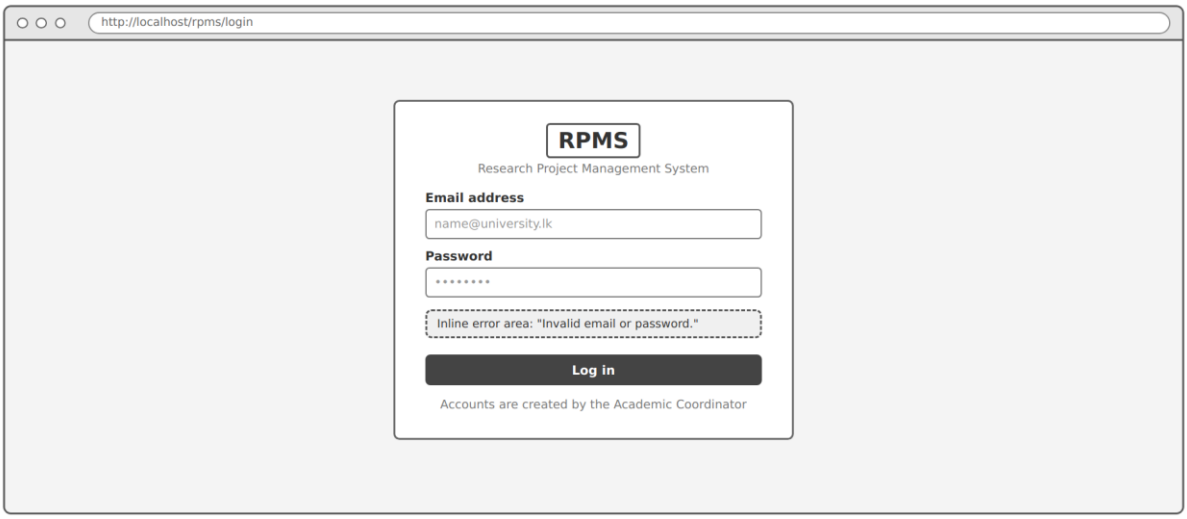


Figure 1.7: *Wireframe — Login page*

The login page (FR-01) is minimal. Errors are shown in a single, non-specific message ("Invalid email or password") so that the page does not reveal which e-mail addresses are registered. There is no self-registration, because accounts are created by the coordinator (FR-04).

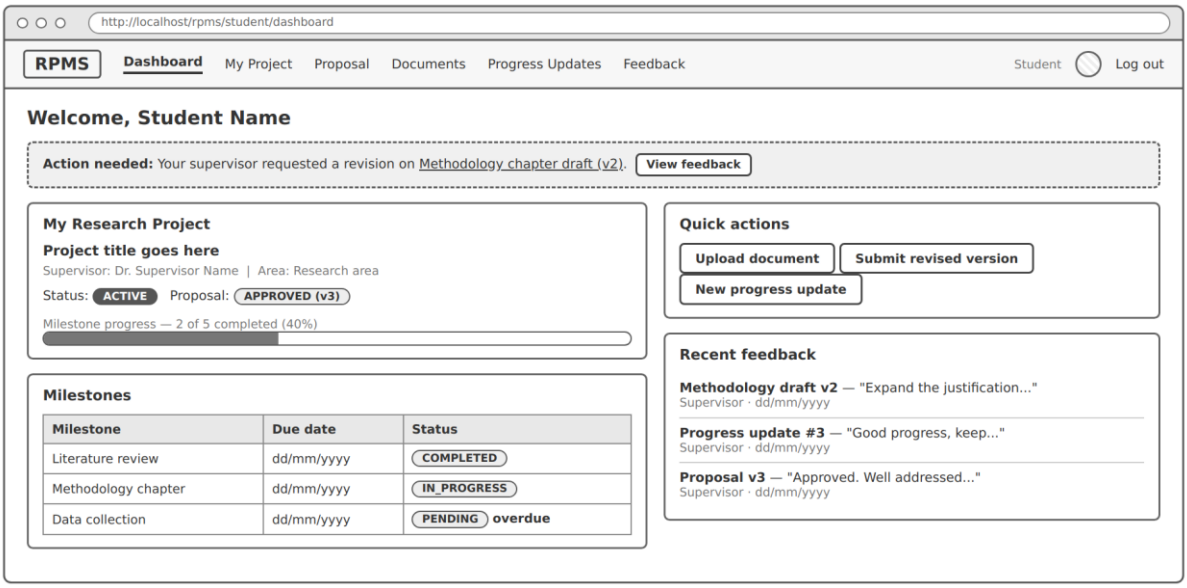


Figure 1.8: *Wireframe — Student dashboard*

The student dashboard (FR-26) answers the question "what do I need to do next?". An *Action needed* banner appears when a revision has been requested, the project and proposal status are shown as badges, and milestones are listed with due dates and an overdue flag (FR-23).

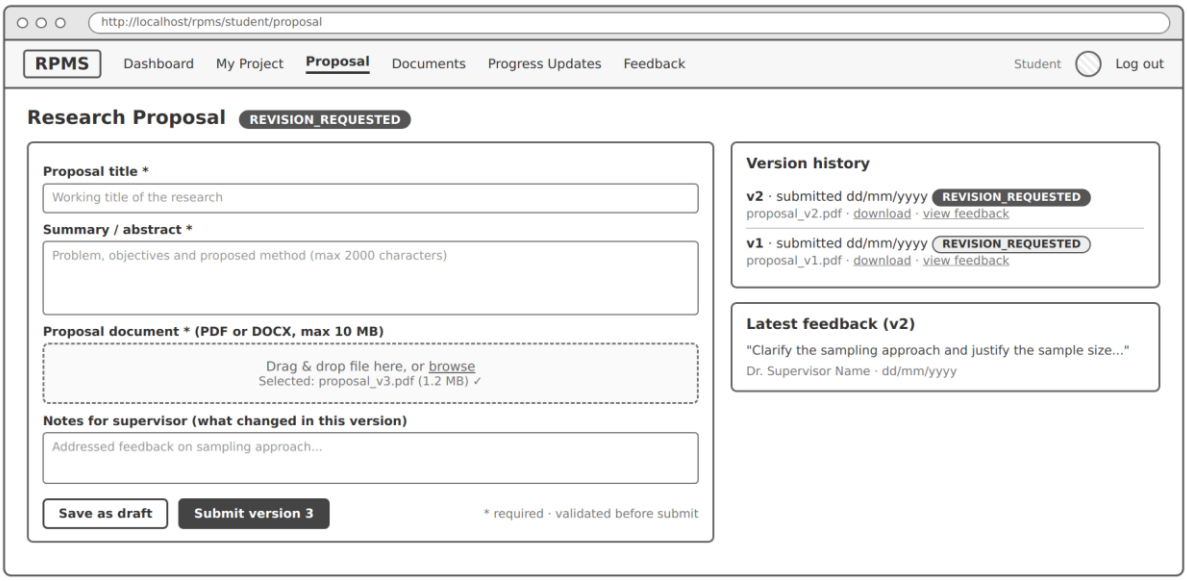


Figure 1.9: *Wireframe — Prepare and submit proposal (UC-01, UC-03)*

The proposal form supports both first submission and resubmission. The version history and the latest feedback are shown beside the form, so the student can address the comments without navigating away. *Save as draft* and *Submit* are separate, clearly labelled actions.

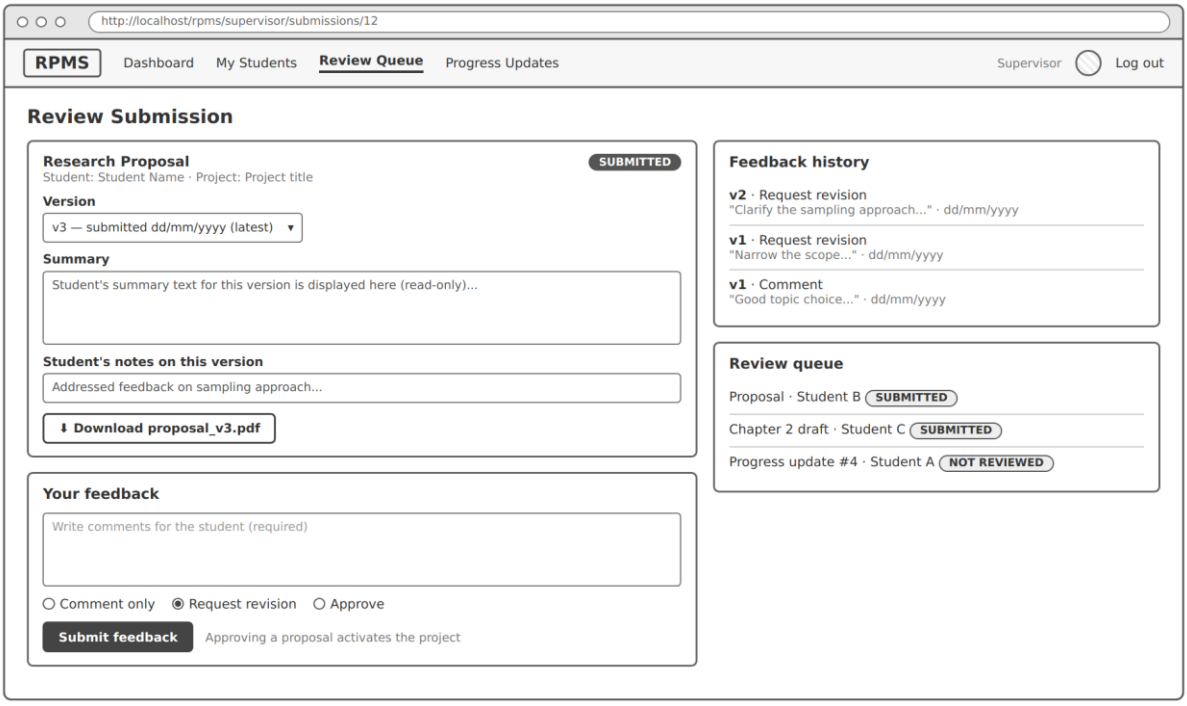


Figure 1.10: *Wireframe — Supervisor review of a submission (UC-02)*

The review screen shows the selected version, the student's change notes and a download button, with the complete feedback history on the right. The feedback form requires a comment and a single decision (comment only, request revision or approve), matching FR-18.

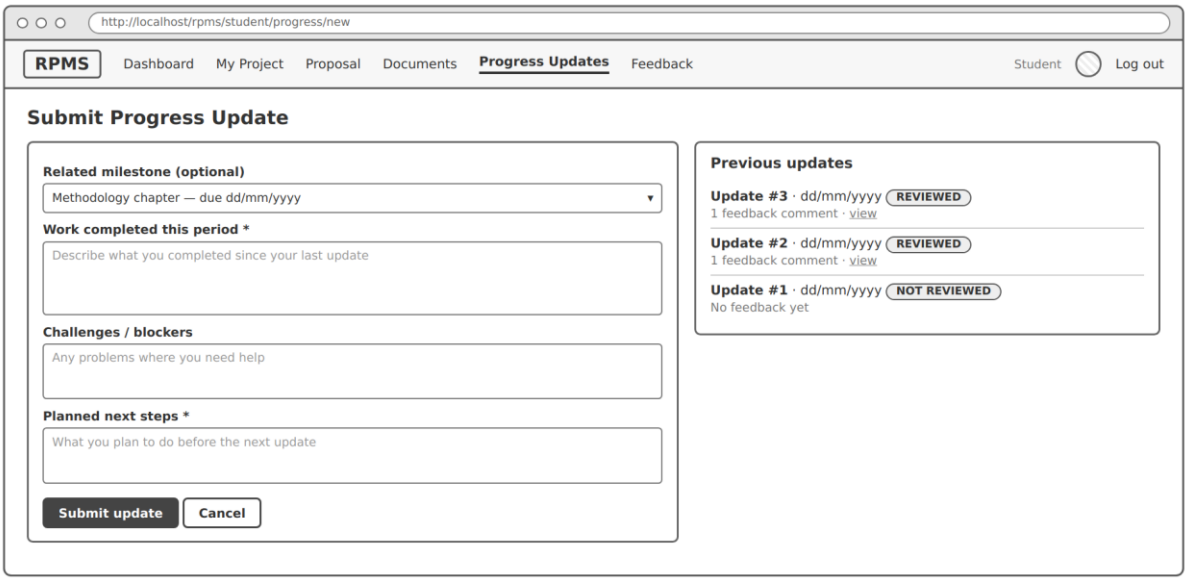


Figure 1.11: *Wireframe — Submit progress update (UC-04)*

The progress update form uses three guided fields (work completed, challenges, next steps) rather than one free-text box, which makes updates consistent and easier for the supervisor to review. Previous updates show whether they have been reviewed.

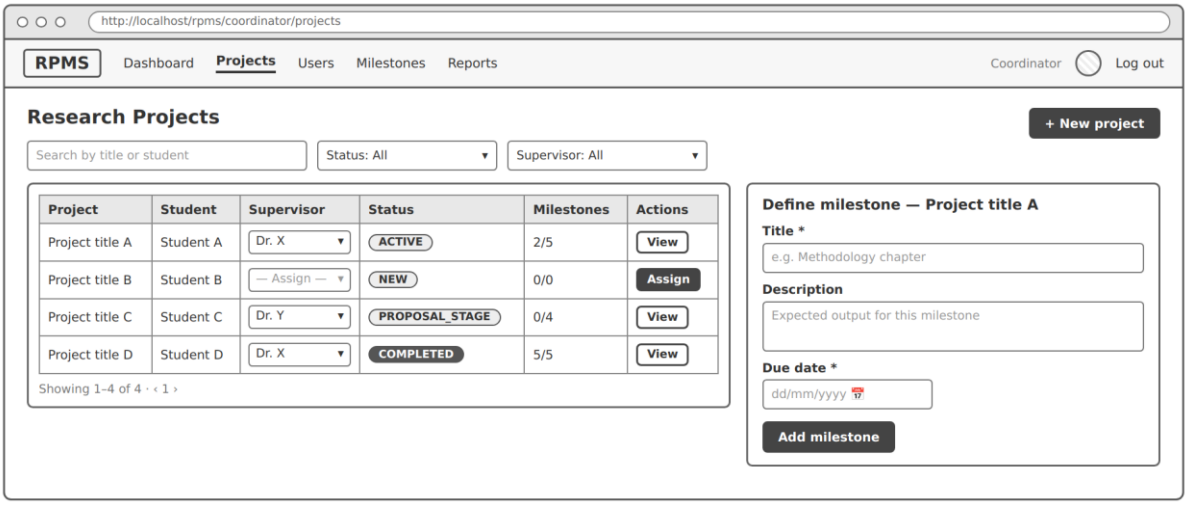


Figure 1.12: *Wireframe — Coordinator project management (UC-05, UC-06)*

The coordinator's project list supports filtering by status and supervisor (FR-27). Unassigned projects show an *Assign* action (UC-05), and milestones can be defined for the selected project in the side panel (UC-06).

Applying usability heuristics

The wireframes were reviewed against Nielsen's (1994) usability heuristics. Table 1.6 shows how the most relevant heuristics are applied.

Table 1.6: *Application of Nielsen's usability heuristics*

Heuristic	How it is applied in the RPMS design
Visibility of system status	Status badges for projects, submissions, milestones and progress updates; the "Action needed" banner; version history on every submission.
Match between system and the real world	The interface uses the vocabulary of research supervision — proposal, milestone, revision, progress update — rather than technical terms.
User control and freedom	Proposals can be saved as drafts and resumed; forms have a Cancel action; nothing is submitted without an explicit button press.
Consistency and standards	The same navigation bar, button styles and badge meanings on every screen of a role; Bootstrap components behave in familiar ways.
Error prevention	Submit is disabled until a supervisor is assigned; file type and size are checked before upload; reassigning a supervisor requires confirmation.
Recognition rather than recall	The latest feedback is shown next to the form used to respond to it, so the student does not need to remember the comments.
Help users recognise and recover from errors	Validation messages appear next to the relevant field in plain language and state how to fix the problem.

1.4 Requirements Traceability

Table 1.7 traces each narrated use case to the functional requirements it satisfies, the database tables it uses and the wireframe that supports it.

Table 1.7: *Traceability from use cases to requirements, data and interface*

Use case	Functional requirements	Main tables	Wireframe
UC-01 Prepare & Submit Proposal	FR-10, FR-11, FR-13, FR-14	submissions, submission_versions	Figure 1.9
UC-02 Review Submission	FR-16, FR-17, FR-18, FR-19	feedback, submissions, projects	Figure 1.10
UC-03 Submit Revised Version	FR-13, FR-14, FR-15, FR-20	submission_versions, submissions	Figure 1.9
UC-04 Submit Progress Update	FR-24, FR-25	progress_updates, milestones	Figure 1.11
UC-05 Assign Supervisor	FR-08	projects, users	Figure 1.12
UC-06 Define Milestones	FR-21, FR-22, FR-23	milestones	Figure 1.12
Log in / View Dashboard	FR-01 – FR-03, FR-26	users, all tables (read)	Figures 1.7, 1.8

References

Cockburn, A. (2001) *Writing Effective Use Cases*. Boston, MA: Addison-Wesley.

Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Harlow: Pearson.

Fowler, M. (2004) *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd edn. Boston, MA: Addison-Wesley.

ISO/IEC (2011) *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Geneva: International Organization for Standardization.

Nielsen, J. (1994) *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. Available at: https://www.nngroup.com/articles/ten-usability-heuristics/ (Accessed: 30 September 2026).

OWASP Foundation (2021) *OWASP Top 10: 2021*. Available at: https://owasp.org/Top10/ (Accessed: 30 September 2026).

PHP Group (n.d.) *PHP Manual*. Available at: https://www.php.net/manual/ (Accessed: 30 September 2026).

Oracle Corporation (n.d.) *MySQL 8.0 Reference Manual*. Available at: https://dev.mysql.com/doc/refman/8.0/en/ (Accessed: 30 September 2026).

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.

University of Colombo School of Computing (2026) *Master of Information Technology — Independent Study (MIT4301): Student Guideline*. Colombo: UCSC.